



Faculty of Engineering and Natural Sciences
Computer Engineering Department

COMPUTER NETWORK CONCEPTS

Term Project Report — Spring 2026

Backgammon Multiplayer Game

A Real-Time Two-Player Network Application

Student Name	Saed O. S. Radi
Student ID	██████████
Project No	2
Instructor	Dr. Samet Kaya
Assistant	Ertuğrul İslamoğlu
Submission Date	May 2026
GitHub Repository	github.com/SaadOsama10/backgammon-network-project
Server IP	██████████ (AWS EC2)

1. Introduction

This report documents the design, implementation, and deployment of a real-time multiplayer Backgammon game. The project demonstrates client-server socket programming, multi-threading, and network communication using Java.

Two players on separate machines connect through an AWS-hosted server and play a fully functional Backgammon game in real time. All game logic is implemented from scratch.

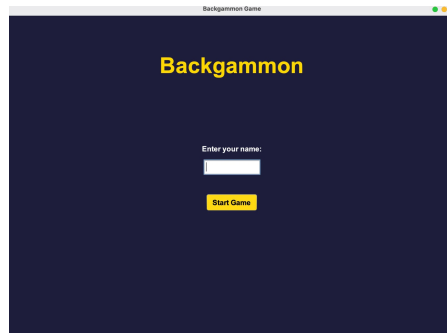


Figure: Application Start Screen — Player Name Input

2. System Architecture

The system follows a three-tier client-server architecture. The server acts as a relay between two client applications, forwarding game moves from one player to the other. All game logic is executed locally on each client, keeping the server lightweight and stateless.

2.1 Architecture Overview

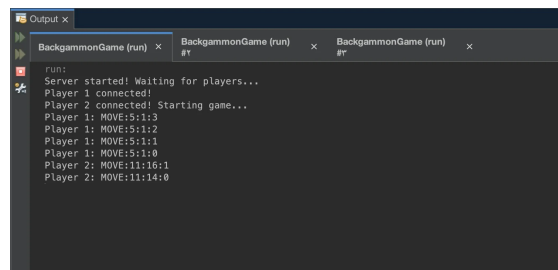
Four main Java classes across three packages:

- `server.GameServer` — deployed on AWS EC2, handles multiple concurrent game sessions
- `client.GameClient` — manages the TCP socket connection and message passing
- `client.GamePanel` — the main game board UI with all game logic
- `client.BoardPanel` — the start screen for entering player name and server IP
- `game.BackgammonBoard` — encapsulates all game rules and board state

2.2 Communication Protocol

Messages are transmitted as plain-text strings over TCP on port 6000:

Message Format	Description
PLAYER:1 / PLAYER:2	Server assigns player number on connection
MOVE:from:to:movesLeft	Regular piece movement (indices 0–23)
MOVE:-1:to:movesLeft	Bar re-entry move (from = -1)
MOVE:from:-2:movesLeft	Bear off move (to = -2)
MOVE:-3:-3:0	Restart game request
DISCONNECT:	Opponent has disconnected



```
run:
Server started! Waiting for players...
Player 1 connected!
Player 2 connected! Starting game...
Player 1: MOVE:5:1:3
Player 1: MOVE:5:1:2
Player 1: MOVE:5:1:1
Player 1: MOVE:5:1:0
Player 2: MOVE:11:16:1
Player 2: MOVE:11:14:0
```

Figure: Server Console Output — Players Connected and Moves Logged

3. Game Logic & Rules

The BackgammonBoard class encapsulates all game state and rules. The board is represented as a 24-element integer array — positive values are Player 1 (white) pieces, negative values are Player 2 (black) pieces, and zero means empty. All logic is executed locally on each client.

3.1 Board Representation

```
private int[] points = new int[24]; // + = white, - = black, 0 = empty
```

3.2 Rules Implemented

- Dice roll — two random values between 1 and 6; rolling doubles grants 4 moves instead of 2
- Move validation — correct direction, exact dice distance, opponent blocking (2+ pieces), and max 5 pieces per point
- Hitting — landing on a single opponent piece (blot) sends it to the Bar
- Bar re-entry — player must re-enter all Bar pieces before any other move; Player 1 enters via points 19–24, Player 2 via 1–6
- Blocked turn — if all re-entry points are blocked by the opponent, the turn is automatically lost
- Bearing off — only allowed when all 15 pieces are in the home board
- Win detection — player wins when all 15 pieces are successfully borne off
- Turn restriction — server-assigned player numbers prevent acting out of turn
- Restart — either player can reset both boards simultaneously

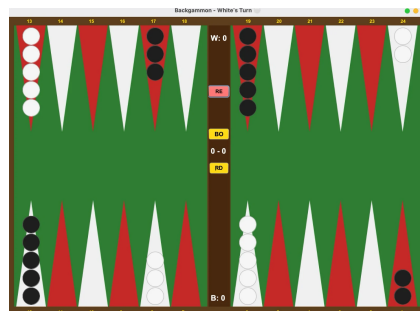


Figure: Game Board — Initial Piece Setup

4. Graphical User Interface

The user interface was built entirely using Java Swing with a custom Null Layout for precise positioning. An innovative image-based approach was chosen for rendering the board triangles.

4.1 Image-Based Triangle Rendering

48 PNG images were pre-generated with Graphics2D — one per combination of triangle color (red/white), piece color, count (0–5), and orientation (normal/flipped). Each triangle point is a JLabel; moves update the icon rather than animating pieces.

4.2 Board Layout

- Left panel (points 13–18 top, 7–12 bottom) — dark green background
- Center bar (50px wide) — dark brown, shows Bar piece counts for both players
- Right panel (points 19–24 top, 1–6 bottom) — dark green background
- All points numbered in gold; numbering runs right-to-left in each quadrant

4.3 Controls

- RD (Roll Dice) — rolls both dice; locked after rolling until a move is made
- BO (Bear Off) — removes a selected piece from the board when bearing off is allowed
- Restart — prompts confirmation then resets the board for both players
- Bar labels (W: / B:) — clickable to select a piece for re-entry from the Bar
- Window title — displays whose turn it is (White's Turn  / Black's Turn )

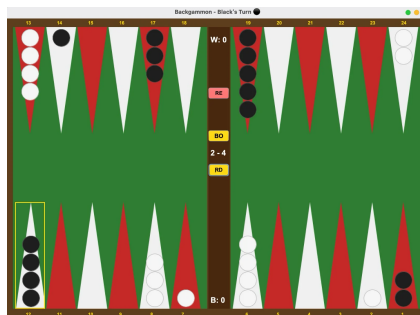


Figure: Gameplay Screenshot — Piece Selected (Golden Border Highlight)

- Resign · player can resign with confirmation; opponent receives a win dialog

5. Network Implementation

The network layer is built on Java's standard socket library (java.net). The design follows a pure relay architecture — the server has no knowledge of game state and simply forwards messages between the two connected clients.

5.1 Server Design

GameServer listens on port 6000. After accepting two players, it spawns one thread per player to read and forward messages. Multiple sessions run simultaneously via nested threads.

```
while (true) { p1=accept(); p2=accept(); new Thread(()->handleGame(p1,p2)).start(); }
```

5.2 Client Design

GameClient connects to the server and starts a background listener thread. It parses MOVE messages and calls applyOpponentMove(). The movesLeft parameter in each message tells the client when to switch the active player.

5.3 Restart & Disconnect Handling

Either player can request a game restart at any time by clicking the Restart button. A confirmation dialog appears first. If confirmed, the client sends a special MOVE:-3:-3:0 message to the server, which forwards it to the opponent. Both clients then reset their board state, dice, and move counters simultaneously · no reconnection needed.



Figure: Restart Confirmation Dialog

If a player disconnects or resigns, the server detects the closed socket and sends a DISCONNECT: message to the remaining player, who receives a win dialog with the option to play again or exit.

6. AWS Deployment

The server runs on an Amazon EC2 instance — publicly accessible with a real IP address as required.

Instance Type	t3.micro (Free Tier eligible)
Operating System	Amazon Linux 2023
Java Version	Amazon Corretto 21
Port	6000 (TCP, open to all IPs via Security Group)
Public IP	[REDACTED]
Region	eu-north-1 (Stockholm)

6.1 Deployment Steps

- Created EC2 instance with a custom Security Group allowing inbound TCP on port 6000
- Installed Java 21 (Amazon Corretto) using yum package manager
- Uploaded GameServer.java to the instance via SCP from local machine
- Compiled the server using javac and launched it using nohup for background execution
- Server is accessible from any network using IP address [REDACTED]

6.2 Starting the Server

```
fuser -k 6000/tcp          # stop any previous instance
javac server/GameServer.java
java server.GameServer     # start in foreground (via browser SSH)
```

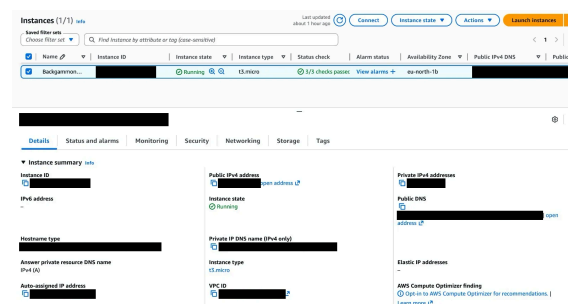


Figure: AWS EC2 Console — Instance Running with Public IP

7. Version Control (GitHub)

The project was tracked using Git from the first day of development. Regular commits were made throughout each development phase to document progress.

Repository URL: github.com/SaadOsama10/backgammon-network-project

- Commits cover all major phases: initial setup, board GUI, game logic, networking, AWS deployment, and bug fixes
- Repository is public, allowing the instructor to review the complete development history
- All source files, images, and the project report are included

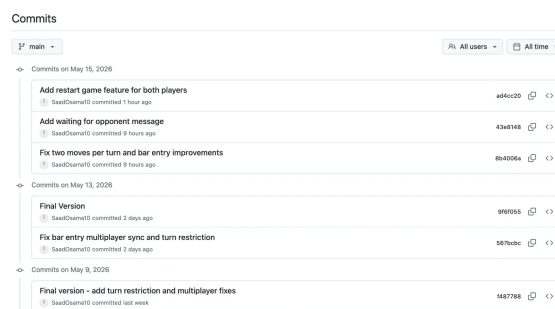


Figure: GitHub Repository — Commit History

8. Challenges & Solutions

Challenge	Solution
Board rendering with dynamic piece counts	Pre-generated 48 PNG images for every state combination instead of real-time drawing
Two-move turns (one per die)	Introduced movesLeft counter; sent with every MOVE message so both sides synchronize turn switching
Bar re-entry sync between clients	Added special from=-1 marker in the MOVE message; applyOpponentMove handles it separately
Player turn enforcement	Server assigns player numbers on connect; client blocks actions when it is not the assigned player's turn
Opponent disconnect notification	Server detects null readLine() and sends DISCONNECT: message to the remaining player
Multiple simultaneous games	Each pair of players is handled in a separate Thread, allowing unlimited concurrent sessions
Resign & disconnect	Server sends DISCONNECT: when socket closes; resign sends MOVE:-4:-4:0 to avoid false disconnect alerts

9. Conclusion

This project successfully implements a real-time multiplayer Backgammon game using Java socket programming. All core requirements were met:

- Two-player multiplayer over TCP with server on AWS EC2
- Complete rule set — dice, bar, bearing off, win detection, doubles
- Turn restriction, restart sync, and disconnect handling
- Multiple concurrent game sessions via server threading
- Java Swing GUI with image-based triangle rendering
- Full project history on GitHub

The project provided hands-on experience with socket programming, multi-threading, client-server architecture, and AWS deployment.